STUDY GUIDE & REFERENCE MANUAL

Study Guide: Agile and Scrum

A Comprehensive Practical Guide to Agile Principles, Scrum Frameworks, and Team Collaboration

Publication Date: October 17, 2026

Table of Contents

1. Introduction to Agile

2. Introduction to Scrum

3. Scrum Roles and Responsibilities

4. Scrum Artifacts and Commitments

5. Scrum Events and Meetings

6. Applying Agile and Scrum in Practice

7. Agile and Scrum Tools and Metrics

8. Advanced Scrum Practices and Challenges

Summary

Answer Key

1. Introduction to Agile

1.1 What Is Agile?

EXPLANATION

Agile is an approach to developing products through short cycles of work, frequent feedback, and ongoing adjustment. Instead of trying to predict every detail before work begins, a team delivers useful pieces of a product, learns from their results, and decides what to do next.

Agile is a set of values and principles rather than one required process. It encourages close collaboration among customers, stakeholders, and the people doing the work. This helps teams respond to changing needs while still focusing on quality and valuable outcomes.

KEY TERMS

- **Iteration:** A short period of work used to produce and review results.
- **Feedback:** Information from users or stakeholders that helps guide future work.
- **Adaptation:** Changing a plan or approach in response to what has been learned.

EXAMPLE

> A team building an online bookstore first releases search and book details. After shoppers try them, the team uses their feedback to improve search before investing in less-used features.

1.2 Traditional vs. Agile Development

EXPLANATION

Traditional, plan-driven development often organizes work into sequential phases such as requirements, design, implementation, and testing. In a Waterfall project, a team may complete and approve much of one phase before moving to the next. This can work well when requirements are stable and extensive upfront coordination is important.

Agile development divides work into smaller batches and revisits plans as the team learns. It can reveal problems earlier and deliver useful functionality sooner, but it requires regular collaboration and does not remove the need for planning or documentation. The best approach depends on uncertainty, constraints, and the needs of the product.

KEY TERMS

- **Waterfall:** A sequential approach in which work typically moves through planned phases.
- **Incremental delivery:** Releasing a product in useful pieces over time.
- **Change cost:** The effort or expense involved in changing a decision or product.

EXAMPLE

> A team designing a new medical device may need formal approvals and carefully controlled phases. A team creating a new café website can release a basic menu first, then adjust the site after seeing how customers use it.

1.3 Agile Manifesto

EXPLANATION

The Agile Manifesto expresses four preferences: individuals and interactions over processes and tools; working software over comprehensive documentation; customer collaboration over contract negotiation; and responding to change over following a plan. The words on the right still have value, but the values on the left receive greater emphasis.

These values encourage teams to communicate directly, demonstrate real product progress, involve customers, and revise plans when new information matters. They are guiding preferences, not rules that require teams to abandon tools, documentation, agreements, or planning.

KEY TERMS

- **Agile Manifesto:** A statement of values and principles for Agile software development.
- **Working software:** Software that functions and can be evaluated in practice.
- **Customer collaboration:** Ongoing partnership with people who understand or use the product.

EXAMPLE

> When a stakeholder requests a change, a team discusses the goal and trade-offs together, updates its plan, and records the important decision rather than insisting that the original plan cannot change.

1.4 Agile Principles

EXPLANATION

The twelve Agile principles expand on the Manifesto's values. They include satisfying customers through early and continuous delivery, welcoming useful changes, delivering working software frequently, and keeping business people and developers in regular contact. They also emphasize motivated people, effective communication, sustainable pace, technical excellence, simplicity, self-organizing teams, and regular reflection.

The principles are meant to guide decisions rather than serve as a checklist that guarantees success. A team can use them to ask whether its way of working is delivering value, supporting quality, and helping people learn. Different products may apply the principles differently because their users, risks, and constraints vary.

KEY TERMS

- **Sustainable pace:** A way of working that can be maintained without relying on repeated overwork.
- **Technical excellence:** Careful engineering that makes a product reliable and easier to change.
- **Self-organizing team:** A team that determines how best to organize its work toward a shared goal.

EXAMPLE

> After discovering that users struggle with checkout, a team releases an improvement promptly, checks the results, and discusses in its next retrospective how to spot usability problems sooner.

✓ Quick Check

- How does Agile use feedback to guide future work?
- What is one important difference between sequential and iterative development?
- Name one Agile Manifesto value and explain its general meaning.

2. Introduction to Scrum

2.1 What Is Scrum?

EXPLANATION

Scrum is a lightweight framework for helping people and teams generate value through adaptive solutions to complex problems. Work is organized into fixed-length Sprints, during which a Scrum Team works toward a shared Sprint Goal and creates a usable Increment that meets its quality standard.

Scrum makes work and progress visible so the team can inspect results and adapt its next steps. It defines accountabilities, events, artifacts, and commitments, but it does not prescribe every task or engineering technique. Teams use appropriate practices to do the work while keeping Scrum's framework understandable.

KEY TERMS

- **Framework:** A structure that guides work without prescribing every detail.
- **Sprint:** A fixed-length period in which the team works toward a valuable outcome.
- **Increment:** A usable step toward the product, integrated with what came before.

EXAMPLE

> A team building a transit app plans a two-week Sprint around helping riders find nearby bus stops, then demonstrates a working map feature and uses rider feedback to plan further improvements.

2.2 Scrum Framework

EXPLANATION

The Scrum framework connects three accountabilities—Product Owner, Scrum Master, and Developers—with a set of events and artifacts. The Product Owner is accountable for product value and the Product Backlog; the Scrum Master is accountable for establishing Scrum; and Developers create a usable Increment each Sprint.

The Sprint contains Sprint Planning, Daily Scrums, the Sprint Review, and the Sprint Retrospective. The Product Backlog, Sprint Backlog, and Increment are the framework's artifacts, with the Product Goal, Sprint Goal, and Definition of Done serving as their commitments. These parts support transparency, inspection, and adaptation.

KEY TERMS

- **Accountability:** A responsibility for an outcome within Scrum.
- **Event:** A formal opportunity to inspect work and adapt.
- **Artifact:** A visible representation of work or value.

EXAMPLE

> At Sprint Planning, a team chooses work linked to a product goal. Developers coordinate daily, stakeholders review the result, and the team then agrees on a process improvement.

2.3 Scrum vs. Agile

EXPLANATION

Agile is a broad set of values and principles. Scrum is one framework that teams can use to put those values into practice. A Scrum Team uses short Sprints, frequent inspection, collaboration, and adaptation, which align with many Agile principles.

Scrum is not the only Agile approach. Kanban, Extreme Programming, and other methods use different structures and practices. A team may also combine Scrum with engineering techniques such as test-driven development. Calling a process Agile does not automatically make it Scrum; Scrum includes specific accountabilities, events, artifacts, and commitments.

KEY TERMS

- **Agile:** A set of values and principles for adaptive product development.
- **Scrum:** A lightweight framework for addressing complex work.
- **Kanban:** An approach that visualizes workflow and manages work in progress.

EXAMPLE

> A product team may use Scrum's Sprints and review events while adopting automated testing practices from Extreme Programming to improve product quality.

2.4 Scrum Terminology

EXPLANATION

Scrum uses terms with specific meanings. A Product is the means of delivering value and can be a service, application, or other outcome. The Product Backlog is an evolving, ordered list of what may improve the product, while the Sprint Backlog contains the Sprint Goal, selected Product Backlog items, and the Developers' plan for delivering them.

An Increment is a usable, verified addition to the product. It must meet the Definition of Done and work with previous Increments. A Sprint is the container for Scrum events and the team's work toward the Sprint Goal. Understanding these terms helps teams discuss work consistently.

KEY TERMS

- **Product:** A vehicle for delivering value to customers or users.
- **Product Backlog:** The ordered, evolving list of work for improving a product.
- **Sprint Backlog:** The Sprint Goal, selected work, and plan for the Sprint.
- **Increment:** A usable addition to the product that meets the Definition of Done.

EXAMPLE

> For a meal-planning app, the Product is the app and service, an item in the Product Backlog could be saving favorite recipes, and that item's completed functionality may become part of an Increment.

✓ Quick Check

- What is Scrum designed to help a team do?
- How does Scrum relate to Agile?
- Which Scrum artifact contains the Developers' plan for the current Sprint?

3. Scrum Roles and Responsibilities

3.1 Product Owner

EXPLANATION

The Product Owner is accountable for maximizing the value of the product resulting from the Scrum Team's work. This includes developing and communicating the Product Goal, creating and clearly communicating Product Backlog items, ordering the backlog, and ensuring that it is transparent and understood.

The Product Owner may do this work personally or delegate tasks, but remains accountable for the outcome. They listen to users, stakeholders, and the team, then make product decisions and clarify priorities. The role is most effective when the Product Owner is accessible and can make timely choices.

KEY TERMS

- **Product value:** The benefit the product provides compared with the effort and resources invested.
- **Product Goal:** The long-term objective for the product.
- **Ordering:** Arranging Product Backlog items to help guide what is considered next.

EXAMPLE

> For a public library app, the Product Owner reviews user feedback and orders improvements so that reliable account access is addressed before a lower-value visual enhancement.

3.2 Scrum Master

EXPLANATION

The Scrum Master is accountable for establishing Scrum as described in the Scrum Guide and for the Scrum Team's effectiveness. They help people understand Scrum, support productive events, and encourage the team to improve how it works.

The Scrum Master serves the team, the Product Owner, and the wider organization. They may help remove impediments or enable people to resolve them, coach stakeholders in effective collaboration, and support organizational changes that make Scrum more effective. The Scrum Master does not assign tasks or act as the team's traditional project manager.

KEY TERMS

- **Impediment:** An obstacle that makes progress or effective work harder.
- **Facilitation:** Helping a group have a productive conversation or event.
- **Servant leadership:** Supporting others' ability to succeed rather than relying on command and control.

EXAMPLE

> When a team is blocked by slow access to a test environment, the Scrum Master works with the relevant department to resolve the access process and helps the team identify ways to prevent the delay from recurring.

3.3 Developers

EXPLANATION

Developers are the people in the Scrum Team who create any aspect of a usable Increment each Sprint. The term describes an accountability, not only software programmers; it can include all the skills needed to build, test, design, and deliver the product.

Developers create the Sprint plan, maintain quality by following the Definition of Done, adapt their plan each day toward the Sprint Goal, and hold one another accountable as professionals. They decide how to do the work and organize their skills to produce the Increment.

KEY TERMS

- **Developers:** Scrum Team members accountable for creating a usable Increment.
- **Sprint Goal:** The single objective that gives the Sprint a purpose.
- **Definition of Done:** The shared description of the quality state required for an Increment.

EXAMPLE

> For a new appointment-booking feature, Developers coordinate user-interface design, programming, accessibility checks, and testing so the feature is usable rather than merely coded.

3.4 The Scrum Team

EXPLANATION

A Scrum Team consists of one Product Owner, one Scrum Master, and Developers. It is a cohesive group focused on one Product Goal at a time. The team is cross-functional, with the skills needed to create value, and self-managing, meaning it decides internally who does what, when, and how.

The entire Scrum Team is accountable for creating a valuable, useful Increment each Sprint. Members collaborate closely and share responsibility for the result rather than treating work as separate handoffs between departments. Scrum Teams are typically small enough to remain connected while still able to complete meaningful work.

KEY TERMS

- **Cross-functional:** Having or accessing the skills needed to complete the work.
- **Self-managing:** Organizing the team's work without an external manager assigning every task.
- **Cohesive:** Working together toward a shared purpose.

EXAMPLE

> A small team with product, design, engineering, and testing skills collaborates on a customer-support portal, coordinating its work instead of passing unfinished tasks between separate departments.

✓ Quick Check

- What outcome is the Product Owner accountable for maximizing?
- How does a Scrum Master support team effectiveness?
- What does it mean for Developers to self-manage their work?

4. Scrum Artifacts and Commitments

4.1 Product Backlog

EXPLANATION

The Product Backlog is an emergent, ordered list of what is needed to improve the product. It is the single source of work undertaken by the Scrum Team. Items may include features, fixes, research, or technical improvements, and their detail can increase as they become more relevant and better understood.

Product Backlog refinement is the ongoing activity of breaking items down and adding details such as descriptions, order, and size. Refinement is not a formal Scrum event. The Product Owner is accountable for the backlog, while the Scrum Team collaborates to make it transparent and useful.

KEY TERMS

- **Product Backlog item:** A piece of work or opportunity to improve the product.
- **Ordering:** Positioning items to communicate their relative importance or sequence.
- **Refinement:** Breaking down and clarifying backlog items as needed.

EXAMPLE

> For a grocery-delivery product, the Product Backlog may include improving address entry, adding order tracking, fixing a payment defect, and exploring support for a new payment provider.

4.2 Sprint Backlog

EXPLANATION

The Sprint Backlog is a plan by and for the Developers. It combines the Sprint Goal, the Product Backlog items selected for the Sprint, and an actionable plan for delivering the Increment. It makes the Developers' current work visible.

The Developers update the Sprint Backlog as they learn more during the Sprint. Its plan can change, but changes should not put the Sprint Goal at risk; the scope can be clarified and renegotiated with the Product Owner as needed. The Sprint Backlog is not a fixed list imposed on the team from outside.

KEY TERMS

- **Sprint Goal:** The objective that explains why the Sprint is valuable.
- **Selected items:** Product Backlog work chosen for the Sprint.
- **Actionable plan:** The Developers' current approach to creating the Increment.

EXAMPLE

> If the Sprint Goal is to let customers track deliveries, Developers might select mapping and status-update items, then adjust task details after discovering how the delivery service provides location data.

4.3 Product Increment

EXPLANATION

An Increment is a concrete stepping stone toward the Product Goal. It adds to all previous Increments, is thoroughly verified, and is usable. Multiple Increments may be created during a Sprint, and the work is not considered part of an Increment unless it meets the Definition of Done.

An Increment must provide value, but it does not have to be released immediately. The Product Owner decides when a release is appropriate, subject to any organizational or regulatory constraints. Keeping the Increment usable helps teams inspect real progress and avoid accumulating unfinished work.

KEY TERMS

- **Usable:** Ready to provide its intended value when put into use.
- **Integrated:** Combined with previous product work rather than isolated from it.
- **Release:** Making a product change available to users or customers.

EXAMPLE

> After completing and testing a password-reset flow to the team's quality standard, the team integrates it with the existing account system; the Product Owner may choose to release it now or coordinate its release with another change.

4.4 Scrum Commitments

EXPLANATION

Each Scrum artifact has a commitment that makes its purpose clearer. The Product Backlog has the Product Goal, the Sprint Backlog has the Sprint Goal, and the Increment has the Definition of Done. These commitments help the team and stakeholders inspect whether work is moving toward a meaningful outcome and meeting an agreed quality standard.

The Product Goal describes a future state for the product and provides a target for planning. The Sprint Goal is the single objective for the Sprint, while allowing flexibility in the specific work. The Definition of Done is a formal description of the state of the Increment when it meets the required quality measures.

KEY TERMS

- **Product Goal:** The long-term objective for the Scrum Team's product.
- **Sprint Goal:** The Sprint's single objective and reason for doing the work.
- **Definition of Done:** The quality criteria work must meet to be part of an Increment.

EXAMPLE

> A team's Product Goal might be to make booking appointments possible online. One Sprint Goal could be enabling customers to choose an available time, with a Definition of Done requiring tested, accessible, integrated functionality.

✓ Quick Check

- How does the Product Backlog differ from the Sprint Backlog?
- What quality condition must work meet to be included in an Increment?
- Which commitment gives the Sprint Backlog its central purpose?

5. Scrum Events and Meetings

5.1 The Sprint

EXPLANATION

The Sprint is a fixed-length event of one month or less that contains all other Scrum events and the work needed to achieve the Product Goal. A new Sprint starts immediately after the previous one ends. Short, consistent Sprints create a regular rhythm for inspection, feedback, and planning.

During a Sprint, the team works toward the Sprint Goal, maintains quality, and adapts its plan as it learns. No changes should endanger the Sprint Goal, quality should not decrease, and the scope can be clarified and renegotiated with the Product Owner. A Sprint may be cancelled only by the Product Owner if its goal becomes obsolete.

KEY TERMS

- **Timebox:** A fixed maximum duration for an event or activity.
- **Sprint Goal:** The objective guiding the work in a Sprint.
- **Sprint cancellation:** Ending a Sprint before its timebox finishes because its goal is obsolete.

EXAMPLE

> A team uses a two-week Sprint to improve online returns. A new finding may change which return-screen details are included, but the team and Product Owner preserve the Sprint Goal of helping customers start a return.

5.2 Sprint Planning

EXPLANATION

Sprint Planning begins the Sprint by establishing a plan for the work ahead. The entire Scrum Team collaborates to discuss why the Sprint is valuable, what can be done, and how the chosen work will be completed. The resulting Sprint Goal, selected Product Backlog items, and initial delivery plan form the Sprint Backlog.

The Product Owner helps participants discuss the most important backlog items and their connection to the Product Goal. Developers select work they believe they can complete while considering capacity and past performance. Planning is a conversation, not a promise that every uncertain task will unfold exactly as predicted.

KEY TERMS

- **Capacity:** The team's available ability to do work during a Sprint.
- **Sprint Backlog:** The goal, selected work, and plan produced through planning.
- **Sprint Planning:** The event that starts a Sprint and establishes its plan.

EXAMPLE

> Before a Sprint, a team reviews its available working days and planned leave, agrees that reducing failed payments is the valuable goal, and selects related work it can reasonably pursue.

5.3 Daily Scrum

EXPLANATION

The Daily Scrum is a 15-minute event for the Developers to inspect progress toward the Sprint Goal and adapt the Sprint Backlog as necessary. It takes place each working day of the Sprint, helping Developers coordinate and identify decisions or obstacles that need attention.

The Daily Scrum is not a status report to a manager, and Scrum does not require a particular speaking format. Developers can use any structure that helps them plan their next workday. Follow-up discussions can happen after the timebox with only the relevant people.

KEY TERMS

- **Daily Scrum:** A short event for Developers to inspect progress and adapt their plan.
- **Sprint Goal:** The shared objective that focuses daily coordination.
- **Impediment:** Something that is obstructing progress or effective work.

EXAMPLE

> Developers notice that two tasks depend on the same API change. They agree during the Daily Scrum who will coordinate the work and update their plan to avoid waiting.

5.4 Sprint Review

EXPLANATION

The Sprint Review is an opportunity for the Scrum Team and stakeholders to inspect the Sprint's outcome and determine future adaptations. Participants discuss progress toward the Product Goal, what changed in the environment, and what to do next. They collaborate on the Product Backlog based on what they learned.

The Review is a working session, not merely a presentation or approval gate. A demonstration of the Increment can support discussion, but the important result is shared understanding and useful decisions about future direction. The event takes place near the end of the Sprint.

KEY TERMS

- **Stakeholder:** A person or group affected by or interested in the product.
- **Sprint Review:** An event to inspect outcomes and adapt future product work.
- **Product Backlog:** The evolving list that may be updated based on review findings.

EXAMPLE

> At a Sprint Review for a school portal, teachers try the new assignment filter and explain that they also need to filter by class. The Product Owner uses this feedback when ordering future backlog work.

5.5 Sprint Retrospective

EXPLANATION

The Sprint Retrospective is an opportunity for the Scrum Team to inspect how the last Sprint went regarding people, interactions, processes, tools, and the Definition of Done. The team identifies assumptions that led it astray and explores what went well, what problems occurred, and how those problems were or were not solved.

The team chooses useful improvements to increase quality and effectiveness. The most impactful changes can be addressed as soon as possible and may be added to the next Sprint Backlog. The event focuses on learning and improvement rather than assigning blame.

KEY TERMS

- **Retrospective:** A team reflection used to identify improvements.
- **Effectiveness:** How well the team is able to achieve valuable outcomes.
- **Experiment:** A small change tried to learn whether a practice improves results.

EXAMPLE

> After repeated testing delays, a team agrees to prepare test data earlier in the next Sprint and checks at the following Retrospective whether that reduced waiting.

✓ Quick Check

- What makes a Sprint a fixed-length event?
- Who uses the Daily Scrum to inspect and adapt the Sprint plan?
- How does a Sprint Review differ from a Sprint Retrospective?

6. Applying Agile and Scrum in Practice

6.1 Creating User Stories

EXPLANATION

A user story is a lightweight way to describe a need from the perspective of someone who uses or benefits from a product. A common template is “As a [type of user], I want [capability], so that [benefit].” The statement helps a team discuss the user's goal without pretending that all details are already known.

User stories are a common Agile practice, not a required Scrum artifact or format. A useful story invites conversation and can be paired with acceptance criteria that clarify important examples or conditions. Teams should focus on the underlying user need rather than splitting work into technical tasks that provide no clear value on their own.

KEY TERMS

- **User story:** A brief description of a user need and its benefit.
- **User perspective:** The viewpoint of the person who will use or benefit from a feature.
- **Acceptance criteria:** Conditions used to clarify and verify expected behavior.

EXAMPLE

> “As a commuter, I want to save a favorite bus stop, so that I can check its next departures quickly” describes a rider's need for a transit app.

6.2 Estimation and Story Points

EXPLANATION

Estimation helps a team discuss the relative size, complexity, effort, and uncertainty of work. Story points are one optional, relative measure; they are not hours and do not represent a universal amount of work. Teams may compare a new item with familiar examples and use a scale such as 1, 2, 3, 5, and 8.

Estimates are uncertain and should not be used to compare teams or judge individual performance. A team can use techniques such as Planning Poker to surface different assumptions. Scrum does not require story points or any particular estimation method; Developers choose an approach that helps them plan.

KEY TERMS

- **Relative estimation:** Comparing the size of work with other work rather than predicting exact time.
- **Story point:** A team's optional unit for representing relative work size.
- **Planning Poker:** A group estimation technique in which participants discuss and reveal estimates.

EXAMPLE

> When estimating a password-reset feature, Developers compare it with a previously completed email-notification change and identify that account security and testing add uncertainty.

6.3 Sprint Execution

EXPLANATION

During a Sprint, Developers carry out and adapt the plan in the Sprint Backlog while keeping the Sprint Goal in view. They collaborate on work, check progress regularly, and integrate completed work so that it meets the Definition of Done. The Sprint Backlog evolves as more is learned.

If new information affects the work, Developers and the Product Owner can clarify or renegotiate scope without endangering the Sprint Goal. A task board or other visualization can help the team see the workflow, but moving a card is not the same as finishing valuable, quality-checked work.

KEY TERMS

- **Sprint execution:** The work of pursuing the Sprint Goal and creating an Increment.
- **Work in progress:** Work that has started but is not yet complete.
- **Integration:** Combining changes into the product so they work together.

EXAMPLE

> While building an order-tracking feature, Developers integrate and test the status view as it is completed, rather than leaving all testing and integration until the final day.

6.4 Scrum Boards and Burndown Charts

EXPLANATION

A Scrum board is a visual tool some teams use to show work and its state, such as “To Do,” “In Progress,” and “Done.” It can make work, dependencies, and bottlenecks easier to discuss. The board is a supporting practice, not a required Scrum artifact, and teams should adapt it to their workflow.

A burndown chart plots remaining work over time, often during a Sprint. It can help a team notice when work is not progressing as expected, but it does not explain why. Such charts should support conversation and adaptation, not be treated as a performance score or a guarantee that scope will finish on a particular date.

KEY TERMS

- **Scrum board:** A visual representation of the team's work and workflow.
- **Burndown chart:** A chart showing remaining work over a period of time.
- **Bottleneck:** A point in a workflow where work accumulates or slows down.

EXAMPLE

> A team sees several review tasks stalled in the “In Progress” column. It discusses the bottleneck and arranges a short pairing session to help complete the reviews.

6.5 Definition of Ready and Acceptance Criteria

EXPLANATION

Acceptance criteria describe specific conditions or examples that help clarify how a backlog item should behave. They support shared understanding and can guide testing, but they do not replace conversation or the Definition of Done. Criteria are most useful when they are clear, testable, and connected to a real need.

A Definition of Ready is a checklist some teams use to assess whether an item is sufficiently understood for discussion or selection. It is not an official Scrum artifact or commitment. Teams should avoid turning it into a rigid gate that prevents learning; they can clarify uncertain work through refinement or select an exploratory task when appropriate.

KEY TERMS

- **Acceptance criteria:** Item-specific conditions describing expected behavior or results.
- **Definition of Ready:** An optional team checklist for assessing backlog-item clarity.
- **Definition of Done:** The shared quality standard required for an Increment.

EXAMPLE

> For a story that lets customers download an invoice, acceptance criteria might specify that the file includes the order total and date; the team's Definition of Done additionally requires accessibility and security checks.

✓ Quick Check

- What user benefit should a well-formed user story communicate?
- Why should story points not be treated as hours or individual performance scores?
- How do acceptance criteria differ from the Definition of Done?

7. Agile and Scrum Tools and Metrics

7.1 Project Management Tools

EXPLANATION

Project management tools help teams organize work, make progress visible, and coordinate conversations. Jira, Trello, and GitHub Projects are examples that can represent backlogs, tasks, workflows, and milestones. The best tool is one that supports the team's actual way of working without adding unnecessary overhead.

Tools do not make a team Agile or guarantee good collaboration. Teams still need clear goals, direct communication, useful feedback, and appropriate product decisions. It is important to keep the tool's information current enough to help people coordinate while avoiding duplicate administrative work.

KEY TERMS

- **Workflow:** The steps work follows from starting to completion.
- **Backlog tool:** Software used to record and organize possible or active work.
- **Information radiator:** A visible display that helps people understand work or progress.

EXAMPLE

> A team uses GitHub Projects to link a customer-facing feature card to its code changes and tests, giving developers and product partners a shared view of progress.

7.2 Velocity

EXPLANATION

Velocity is a planning measure some teams calculate from the amount of estimated work completed in a Sprint. If a team uses story points, it may look at several Sprints of completed points to understand its own recent pattern and inform future forecasts.

Velocity is not a Scrum requirement, a productivity measure, or a target that should be increased for its own sake. It depends on a team's estimation approach and Definition of Done, so comparisons between teams are misleading. Teams should use historical data cautiously and adjust plans when capacity or conditions change.

KEY TERMS

- **Velocity:** A team's historical measure of completed estimated work per Sprint.
- **Forecast:** An informed estimate of possible future delivery, not a guarantee.
- **Capacity:** The team's available ability to do work in a given period.

EXAMPLE

> Before planning a holiday-period Sprint with several people away, a team reviews its own recent delivery history and reduces its forecast rather than expecting a typical Sprint's output.

7.3 Burndown and Burnup Charts

EXPLANATION

A burndown chart shows how much work remains over time. A burnup chart shows completed work over time and may also display the total scope. Both can help teams and stakeholders notice trends, discuss progress, and investigate changes.

Charts are simplified views of a changing situation. A burndown can look behind when estimates or scope change, while a burnup can show scope growth more explicitly. Neither chart explains the cause of a trend or replaces conversation about value, quality, and risks.

KEY TERMS

- **Burndown chart:** A visual trend of remaining work over time.
- **Burnup chart:** A visual trend of completed work, often alongside total scope.
- **Scope change:** An adjustment to the amount or content of work being considered.

EXAMPLE

> A product team uses a burnup chart to show that it completed checkout improvements while the total scope grew after a newly discovered payment requirement.

7.4 Agile Quality Practices

EXPLANATION

Agile teams need technical practices that help them deliver working, reliable product changes frequently. Code reviews let peers examine changes; automated tests check expected behavior; and continuous integration frequently combines changes and runs checks to detect problems early.

Continuous delivery is the ability to keep software in a state where it can be released reliably, while continuous deployment automatically releases changes that meet the required checks. These practices are complementary but distinct. Their purpose is to reduce avoidable risk and support a usable Increment, not to replace thoughtful engineering or user feedback.

KEY TERMS

- **Code review:** Examination of a code change by one or more peers.
- **Continuous integration (CI):** Frequent integration of changes with automated verification.
- **Continuous delivery:** Keeping software ready for a reliable release.
- **Continuous deployment:** Automatically releasing changes that pass the delivery pipeline.

EXAMPLE

> When a developer updates a shopping cart, automated tests run on the change and a peer reviews it before it is integrated into the product.

✓ Quick Check

- What kinds of work can a project management tool help a team make visible?
- Why is velocity most useful when interpreted as a team's own historical information?
- What is the difference between continuous delivery and continuous deployment?

8. Advanced Scrum Practices and Challenges

8.1 Managing Changing Requirements

EXPLANATION

Agile teams expect that understanding of a product may change as users try it, markets shift, or new risks emerge. They use feedback and evidence to revisit priorities and adapt the Product Backlog rather than assuming every early requirement is permanently correct.

Change still needs thoughtful management. The Product Owner considers value and trade-offs, while the Scrum Team clarifies the work and its effect on goals. During a Sprint, scope can be clarified and renegotiated with the Product Owner as more is learned, but changes should not endanger the Sprint Goal or lower the quality of the Increment.

KEY TERMS

- **Requirement:** A capability, outcome, or constraint that a product or service needs to address.
- **Trade-off:** A choice that balances competing needs, costs, or risks.
- **Feedback loop:** A repeated cycle of trying work, learning from results, and adapting.

EXAMPLE

> User testing shows that shoppers abandon a complicated delivery form. The Product Owner orders simpler address entry ahead of a planned visual enhancement, and the team explores the change in upcoming work.

8.2 Handling Common Scrum Challenges

EXPLANATION

Teams may struggle with unclear goals, overloaded Sprints, work that is frequently interrupted, meetings that feel unhelpful, or low trust between collaborators. These symptoms can have different causes, so the team should make the problem visible and investigate it rather than applying a one-size-fits-all fix.

The Scrum Team can use its events to improve: clarify the Product Goal and Sprint Goal, discuss realistic capacity, make impediments visible, and select a specific improvement in the Retrospective. Stakeholders can also help by providing timely decisions and respecting the team's focus while remaining available for collaboration.

KEY TERMS

- **Unclear goal:** An objective that does not give people a shared understanding of the intended outcome.
- **Overcommitment:** Taking on more work than the team can reasonably complete.
- **Impediment removal:** Reducing an obstacle that interferes with effective progress.

EXAMPLE

> After several Sprints end with unfinished work, the team examines interruptions and dependencies in its Retrospective, then tries limiting concurrent work and making urgent requests visible.

8.3 Scaling Scrum

EXPLANATION

When several teams work on one product, coordination is needed to align their efforts, manage dependencies, and maintain an integrated product. Teams need a shared Product Goal and Product Backlog, clear communication, and a compatible Definition of Done so the result can work together.

Frameworks such as Nexus or Scrum@Scale offer ways to coordinate multiple teams, but adding a scaling framework does not remove the need to inspect and adapt. Organizations should choose the lightest approach that addresses their actual coordination challenges. More teams can add communication paths and dependencies, so simplifying the product or reducing unnecessary handoffs may help.

KEY TERMS

- **Scaling:** Coordinating multiple teams working toward outcomes for one product or related products.
- **Dependency:** Work or a decision that relies on another piece of work or team.
- **Integrated Increment:** A product result that combines the teams' work and meets the shared quality standard.

EXAMPLE

> Two teams building a travel service agree on a shared Definition of Done and coordinate changes to the booking API so their features can be integrated into one working product.

8.4 Continuous Improvement

EXPLANATION

Continuous improvement means regularly looking for ways to increase product value, quality, and team effectiveness. Scrum's inspection and adaptation cycle gives teams recurring opportunities to use feedback, examine their work, and adjust their approach.

Improvement works best when it is specific and observable. A team can choose a small experiment, agree on what it hopes to learn, and revisit the result rather than trying many changes without checking their effects. Improvements can concern product behavior, collaboration, engineering practices, or the way work flows.

KEY TERMS

- **Continuous improvement:** Ongoing efforts to improve outcomes and ways of working.
- **Experiment:** A deliberate trial used to test whether a change helps.
- **Outcome:** A meaningful result or effect, rather than simply completed activity.

EXAMPLE

> A team wants to reduce defects found after release, so it adds a brief exploratory testing session before each Increment is considered done and reviews the results after two Sprints.

8.5 Agile and DevOps

EXPLANATION

DevOps is a set of cultural and technical practices that brings development and operations work closer together. It emphasizes shared responsibility, automation, reliable systems, and learning from how software behaves in production. Agile and DevOps can complement each other by connecting iterative product planning with dependable ways to build, release, and operate software.

Agile practices help teams decide what valuable change to pursue and learn from feedback; DevOps practices such as automated deployment, monitoring, and infrastructure automation can help deliver and support that change. Neither approach is only a toolset, and adopting automation without collaboration or operational learning does not by itself create effective DevOps.

KEY TERMS

- **DevOps:** Practices that improve collaboration and delivery between development and operations.
- **Monitoring:** Observing a running service to understand its health and behavior.
- **Automation:** Using software to perform repeatable tasks consistently.

EXAMPLE

> After releasing a search improvement, a team monitors search speed and error rates, then uses that production information to decide whether a follow-up change is needed.

✓ Quick Check

- **How can a team respond to new requirements without losing focus during a Sprint?**
- **What shared practices help multiple Scrum Teams integrate their work?**
- **How do Agile and DevOps practices support different parts of product delivery?**

Summary

- **Agile Core Values and Mindset:** Agile emphasizes individuals and interactions, working software, customer collaboration, and responsiveness to change over rigid processes and heavy documentation, delivering customer value in short, iterative cycles.
- **The Scrum Framework:** Scrum is a lightweight, empirical framework designed to solve complex problems through iterative work cycles called Sprints (lasting one month or less), culminating in a usable, quality-checked Increment.
- **Scrum Accountabilities:** A cohesive, self-managing Scrum Team comprises three distinct accountabilities: the Product Owner (maximizes product value and manages the Product Backlog), the Scrum Master (champions Scrum effectiveness and removes impediments), and Developers (build the usable Increment).
- **Artifacts and Commitments:** The three Scrum artifacts ensure transparency and focus through explicit commitments: the Product Backlog commits to the Product Goal, the Sprint Backlog commits to the Sprint Goal, and the Increment commits to the Definition of Done.
- **Timeboxed Empirical Events:** Sprints provide a regular cadence consisting of Sprint Planning, Daily Scrums, Sprint Reviews, and Sprint Retrospectives, enabling frequent inspection and adaptation without unnecessary meeting churn.
- **Practical Application and Estimation:** Effective teams employ user stories, clear acceptance criteria, and relative estimation techniques (such as story points) to foster shared understanding, maintain a sustainable pace, and avoid treating estimates as fixed deadlines.
- **Technical Excellence and DevOps Alignment:** Sustainable agility depends on solid engineering practices—including code reviews, automated testing, continuous integration, and continuous delivery—to maintain reliable, deployable codebases.
- **Continuous Improvement and Scaling:** Agile maturity thrives on inspecting workflows, executing small improvements, and, when multiple teams collaborate, scaling through unified product backlogs, clear cross-team dependencies, and aligned quality standards.

Answer Key

Section 1: Introduction to Agile

How does Agile use feedback to guide future work?

> Agile teams deliver work in short, incremental cycles, gather feedback from users and stakeholders on functional increments, and use those insights to adjust upcoming priorities and refine plans.

What is one important difference between sequential and iterative development?

> Sequential (Waterfall) development moves through fixed, approved phases in linear order, whereas iterative development delivers small functional batches repeatedly, learning from each release and adapting continuously.

Name one Agile Manifesto value and explain its general meaning.

> *Responding to change over following a plan:* While planning provides essential direction, teams prioritize adapting to new learnings, stakeholder feedback, and shifting realities over rigidly following an outdated plan.

Section 2: Introduction to Scrum

What is Scrum designed to help a team do?

> Scrum is designed to help people, teams, and organizations generate value through adaptive solutions to complex problems.

How does Scrum relate to Agile?

> Agile is a broad philosophy of values and principles, whereas Scrum is a specific, lightweight framework that puts Agile principles into practice through defined accountabilities, events, and artifacts.

Which Scrum artifact contains the Developers' plan for the current Sprint?

> The Sprint Backlog (which includes the Sprint Goal, selected Product Backlog items, and the actionable plan for delivering the Increment).

Section 3: Scrum Roles and Responsibilities

What outcome is the Product Owner accountable for maximizing?

> The Product Owner is accountable for maximizing the value of the product resulting from the work of the Scrum Team.

How does a Scrum Master support team effectiveness?

> The Scrum Master supports effectiveness by establishing Scrum as described in the Scrum Guide, facilitating events, coaching the team and organization, and helping remove impediments.

What does it mean for Developers to self-manage their work?

> Self-managing means Developers internally decide who does what, when, and how to create a usable Increment, without an outside manager directing or assigning their daily tasks.

Section 4: Scrum Artifacts and Commitments

How does the Product Backlog differ from the Sprint Backlog?

> The Product Backlog is an evolving, ordered inventory of everything that could improve the product long-term (managed by the Product Owner). The Sprint Backlog is the Developers' tactical plan and selected items dedicated specifically to the current Sprint.

What quality condition must work meet to be included in an Increment?

> Work must satisfy the team's agreed Definition of Done.

Which commitment gives the Sprint Backlog its central purpose?

> The Sprint Goal.

Section 5: Scrum Events and Meetings

What makes a Sprint a fixed-length event?

> A Sprint has a predetermined, non-extendable timebox of one month or less; it ends on a fixed date regardless of work completed, and a new Sprint begins immediately after.

Who uses the Daily Scrum to inspect and adapt the Sprint plan?

> The Developers.

How does a Sprint Review differ from a Sprint Retrospective?

> The Sprint Review inspects the product Increment with stakeholders to adapt the Product Backlog, while the Sprint Retrospective inspects internal team dynamics, processes, and tools to identify team-level improvements.

Section 6: Applying Agile and Scrum in Practice

What user benefit should a well-formed user story communicate?

> It communicates the underlying need and the specific benefit or value the user aims to achieve (the 'so that [benefit]' clause), rather than just technical implementation tasks.

Why should story points not be treated as hours or individual performance scores?

> Story points are relative, uncertain estimates of effort and complexity used for team forecasting; they do not correlate to clock hours and cannot be used to compare individual performance or different teams.

How do acceptance criteria differ from the Definition of Done?

> Acceptance criteria define specific, testable requirements for an individual backlog item, whereas the Definition of Done applies globally across all items to ensure the quality and completeness of an Increment.

Section 7: Agile and Scrum Tools and Metrics

What kinds of work can a project management tool help a team make visible?

> Backlogs, active tasks, workflow stages, dependencies, bottlenecks, milestones, and work-in-progress status.

Why is velocity most useful when interpreted as a team's own historical information?

> Velocity reflects an individual team's specific sizing scale, capacity, and Definition of Done, making it useful solely for internal forecasting rather than external productivity comparisons.

What is the difference between continuous delivery and continuous deployment?

> Continuous delivery ensures that code changes are automatically tested and kept in a release-ready state for manual deployment decisions; continuous deployment automatically releases every verified change directly to production.

Section 8: Advanced Scrum Practices and Challenges

How can a team respond to new requirements without losing focus during a Sprint?

> By evaluating them against the Sprint Goal: if they align with the goal, scope can be renegotiated with the Product Owner; if they are unrelated, they are added to the Product Backlog for future prioritization without disrupting the current Sprint.

What shared practices help multiple Scrum Teams integrate their work?

> Sharing a single Product Goal and Product Backlog, maintaining a compatible Definition of Done, and coordinating dependencies to deliver a single integrated Increment.

How do Agile and DevOps practices support different parts of product delivery?

> Agile practices guide iterative product definition and feedback-driven prioritization, while DevOps practices automate build, testing, deployment, and monitoring to ensure reliable, continuous operations in production.